A VISUAL REFERENCE GUIDE · 2026

The Complete AI Ecosystem

Data, Models, Local AI, Infrastructure, Applications, and Responsible AI

01 02 03 04 05 06 07 08 09 10 11 12 13 14 15

15 connected layers · from raw data to production, governed end to end

MASTER ECOSYSTEM MAP · 15 LAYERS

How the 15 layers connect

01
Data sources

02
Ingestion & integration

03
Storage & retrieval

04
Quality, prep & data governance

05
AI & model development

06
Model families & capabilities

12
Foundation, open-weight & local models

08
Deployment & inference

07
AI application architecture

13
Connected systems

14
People, users & industries

09
MLOps, LLMOps & observability — the feedback loop back to 01

10
Compute, cloud, hardware & infrastructure

11
AI tools & developer ecosystem

15 · Governance, safety, security & responsible AI — a continuous control layer across every layer above, never a final step

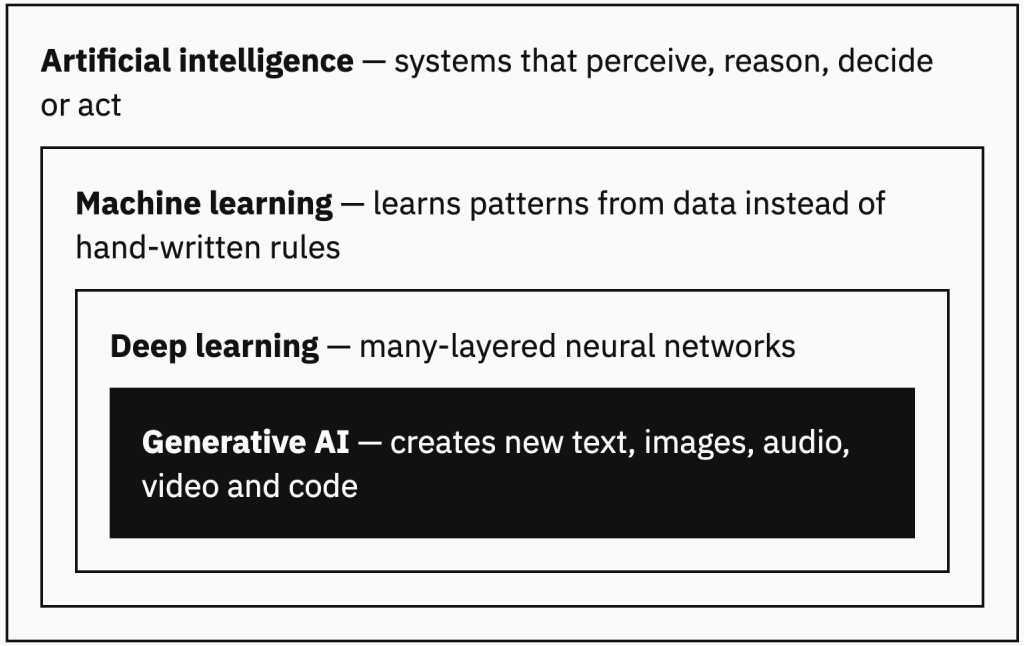
SECTION 01

Foundations

The vocabulary of AI, and the parts every AI system is built from.

§1 FOUNDATIONS

Understanding the AI landscape



Foundation models

Large models pre-trained on broad data, then adapted to many tasks. Most generative AI is built on them.

LLMs and SLMs

Large language models maximize capability. Small language models, typically a few billion parameters, trade breadth for speed, cost and on-device use.

Traditional vs generative

Traditional ML predicts or classifies — a fraud score, a demand forecast. Generative AI produces new content. Real systems often combine both.

Takeaway — the terms nest; they are not synonyms. All generative AI is deep learning, but much production AI is still classical machine learning.

AI system architecture

Seven parts: data · models · orchestration · inference · application · users · governance

Concept	What it is	Example
Model	Trained weights that turn inputs into outputs	An open-weight LLM file; a fraud classifier
Model API	A hosted endpoint that runs a model for you, billed per use	A provider’s chat or messages endpoint
AI application	Software wrapping a model with prompts, data, interface and business rules	A support chatbot; a meeting summarizer
AI agent	An application where the model plans and acts through tools, in a loop	Triages tickets and updates the CRM
AI platform	Shared infrastructure to build, serve, evaluate and govern many models and apps	Managed hosting with evals, monitoring and access control

Takeaway — a model is a component, not a product. The application around it decides most of the quality, cost and risk.

THE COMPLETE AI ECOSYSTEM

SECTION 02 · LAYERS 01–04

Data & Knowledge

Where AI gets its raw material, and how that material becomes usable.

Data sources · Ingestion · Storage · Retrieval stores · Preparation

Data sources and collection

AI consumes five shapes of data: structured, semi-structured, unstructured, streaming and multimodal.

Structured

MySQL · PostgreSQL · SQL Server · data warehouses · enterprise records

Semi-structured

MongoDB · Cassandra · DynamoDB · JSON documents · application logs

Unstructured

Websites · PDFs · books · papers · code & docs · knowledge bases

Multimodal

Images · video · audio · speech · scanned documents

Streaming

APIs · telemetry · event streams · IoT sensors · edge systems

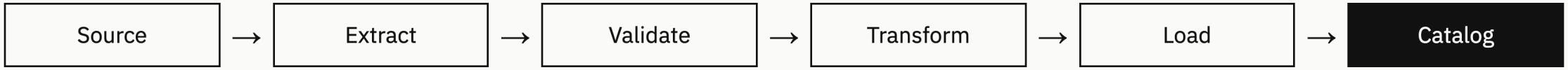
Human & synthetic

User content & feedback · open datasets · synthetic data · simulations

Takeaway — know each source's owner, license and refresh rate before it feeds a model; those three decide what you may build on it.

Data ingestion, ETL/ELT and streaming

Ingestion moves data from sources into storage, checking it on the way in.



Batch

Scheduled bulk loads. ETL transforms before loading; ELT loads first and transforms in the warehouse. Spark · Airflow · dbt

Real-time

Streaming, Change Data Capture from databases, event-driven pipelines. Apache Kafka · Apache Flink

Connectors

API integrations, web crawling, document ingestion. AWS Glue · Azure Data Factory · Google Cloud Dataflow

Controls

Schema enforcement, validation, deduplication, data contracts, lineage and provenance

Takeaway — reject bad data at the door. Every record that slips past validation can resurface later as a wrong answer.

Data lakes, lakehouses, warehouses and databases

Store	Best for	Examples
Object storage	Raw files, model artifacts, backups at any scale	Amazon S3 · Azure Blob · Google Cloud Storage
Lake / lakehouse	Open-format files with table features and big compute	Databricks · Apache Iceberg tables
Data warehouse	Governed analytical SQL and BI	Snowflake · BigQuery · ClickHouse
Relational / NoSQL	Exact, transactional application records	PostgreSQL · MySQL · MongoDB · Redis
Time-series	Metrics and sensor readings over time	TimescaleDB · InfluxDB
Feature store	The same ML features for training and serving	Feast
Metadata catalog	Discovery, ownership, lineage, access policy	Unity Catalog · DataHub

Takeaway — pick storage by access pattern: files, analytics or transactions. Retention rules and access control apply to all of them.

Vector, search, graph and SQL stores are not interchangeable

Each answers a different kind of question. Most RAG systems combine two or more.

Vector database

Finds similar meaning

“passages about refund disputes”

Qdrant · Milvus · Weaviate · pgvector

Full-text search

Finds exact words, ranked

“error code E-4031”

Elasticsearch · OpenSearch

Graph database

Finds relationships across hops

“which suppliers feed recalled products?”

Neo4j · Amazon Neptune · knowledge graphs

Relational / SQL

Finds exact records and totals

“refunds in March by region”

PostgreSQL · MySQL

Takeaway — hybrid search (vector + keyword) is the common default; object storage keeps the original files these indexes point to.

Data quality, preparation and governance

Clean
Normalize, deduplicate, test quality

Shape
Tokenize, label, engineer features

Index
Parse, chunk, generate embeddings

Protect
Redact PII, control access, retain

Trust
Check bias, license, provenance

Split
Train, validation, test; version it

Purpose	Data needed	Key preparation
Classical ML	Labeled tables	Features, labels, leak-free splits
Foundation pre-training	Trillions of tokens	Filtering, deduplication, contamination checks, licensing
Fine-tuning	Curated examples	Instruction and preference pairs, PII removal, versioning
RAG	Your documents	Parsing, chunking, embeddings, metadata and access tags

Takeaway — preparation depends on the goal: training needs scale and cleanliness; RAG needs good parsing and permissions.

THE COMPLETE AI ECOSYSTEM

SECTION 03 · LAYERS 05, 06, 12

Models & Training

How models are built, adapted, chosen and accessed.

Development · Training pathways · RAG vs fine-tuning · Model families · Multimodal · Model access

The model development lifecycle

01
Define
Problem, success metric and a simple baseline

02
Explore
Statistical analysis, EDA, data checks

03
Engineer
Features and labels, or prompts and context

04
Train or adapt
Supervised, unsupervised, self-supervised or RL

05
Tune
Hyperparameter search, tracked experiments

06
Evaluate
Held-out tests, task evals, safety and bias checks

07
Register
Approval, model registry, versioned artifacts

08
Optimize
Distill or quantize, then hand off to serving

Takeaway — start from a baseline and a metric. Without them you cannot tell whether a bigger model or more training actually helped.

Six ways to get a working model

Workflow	Compute	Data	Watch for
Train from scratch	Large GPU or TPU clusters, weeks to months	Billions to trillions of tokens	Checkpointing, evals at scale, data licensing
Full fine-tune	Several GPUs, hours to days	Thousands or more examples	Base-model license, forgetting old skills
PEFT: LoRA, QLoRA	One or a few GPUs	Hundreds to thousands of examples	Small adapters; QLoRA trains on a 4-bit base
RAG	Embedding and inference only	Your documents	Retrieval quality, freshness, permissions
Prompt-only	Inference only	A few examples	Fastest start; capped by the base model
Distill, quantize	Teacher inference plus student training	Teacher outputs	Quality loss; terms on using teacher outputs

Prompting, RAG, fine-tuning or long context?

	Prompting	RAG	Fine-tuning	Long context
Changes	Instructions only	Knowledge at query time	Behavior, format, style	How much fits in one request
Best for	Quick, general tasks	Private, changing facts with citations	Consistent output and domain skills	A few large documents
Freshness	Instant	As fresh as the index	Frozen at training time	Per request
Cost	Lowest	Indexing plus retrieval	Training run plus hosting	Many tokens per call
Main risk	Brittle prompts	Poor retrieval, wrong answers	Stale facts, lost skills	Cost; details lost mid-context

Takeaway — use RAG for what the model should know, fine-tuning for how it should behave. They combine well.

Model families: general-purpose and specialized

General-purpose

Large language models
Small language models
Multimodal models
Code-generation models

One model, many tasks, steered by prompts.

Perception

Vision · detection & segmentation · speech recognition

Generation

Text-to-speech · image · video · diffusion models

Retrieval

Text and image embeddings · rerankers

Predictive

Classification · regression · forecasting · anomaly detection

Ranking & graph

Recommendation & ranking · graph ML

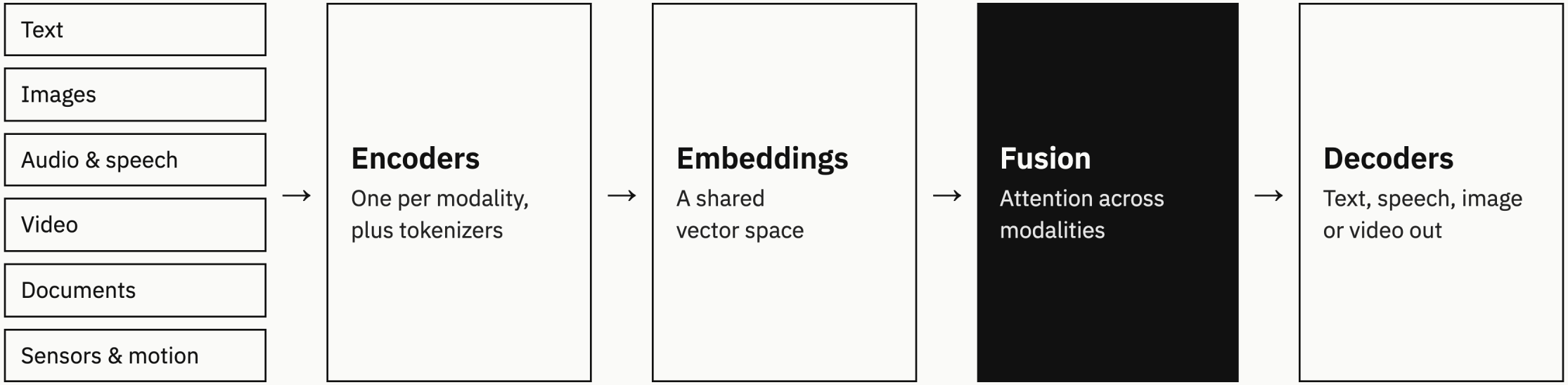
Physical

Robotics & action · geospatial · scientific

Takeaway — specialized models are usually smaller, cheaper and more accurate on their one task; general models trade that for breadth.

Multimodal AI

Encoders turn each input into tokens or embeddings that one model can reason over.



Takeaway — support varies by model: many read images but cannot produce them. Check each model's input and output modalities.

Four ways to access a foundation model

	A · Proprietary API	B · Open-weight	C · Local	D · Cloud-hosted open
Examples	OpenAI GPT · Anthropic Claude · Google Gemini	Llama · Qwen · Mistral · DeepSeek · Gemma · Phi	Ollama · LM Studio · llama.cpp · MLX	Bedrock · Microsoft Foundry · Vertex AI · Together
Runs on	The provider’s cloud	Wherever you choose	Your own machine	A managed host or your cloud GPUs
Pricing	Per token	Free weights; you pay compute	Hardware you own	Per token or GPU-hour
Privacy	Provider terms apply	Full control of weights	Data stays on device	Depends on the host
Watch for	Lock-in, deprecations	License terms vary	RAM and VRAM limits	Region, SLAs, egress

Takeaway — open-weight describes the license, not the location. The same model can run on a laptop or in a data center.

Open-weight model families and their licenses

Family	Developer	Typical license	Notes
Llama	Meta	Llama community license (custom)	Use policy; extra terms for huge services
Qwen	Alibaba	Mostly Apache 2.0	Wide size range; multilingual, coding
Mistral	Mistral AI	Apache 2.0 for many; others custom	Check each model’s terms
DeepSeek	DeepSeek	MIT for recent releases	Reasoning; mixture-of-experts
Gemma	Google	Gemma terms of use (custom)	Small to mid sizes; on-device
Phi	Microsoft	MIT	Small language models
gpt-oss	OpenAI	Apache 2.0	Open-weight reasoning models
OLMo	Ai2	Apache 2.0	Fully open: weights, data, code

SECTION 04 · LAYER 12C

Local AI

Running models on your own laptops, workstations and servers.

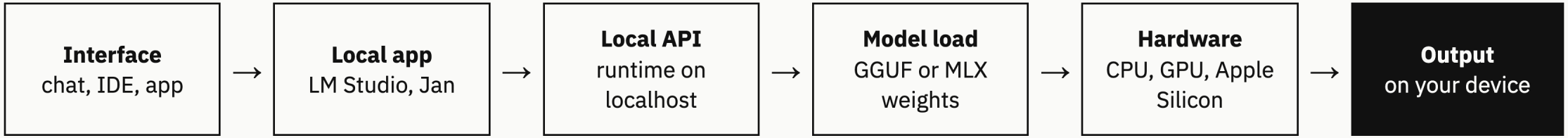
Tools and runtimes · Local architecture · Hardware and memory

Local AI tools play different roles

Tool	Role	Best for
LM Studio	Desktop app	Browse, download and chat with models
Jan · GPT4All	Desktop apps	Private offline chat, local documents
Ollama	Model manager, local API	One-command models with a localhost API
LocalAI	Local API server	OpenAI-compatible API over many backends
llama.cpp	Inference runtime	GGUF models on CPU, Metal, CUDA, ROCm
MLX	Apple Silicon runtime	Fast inference and fine-tuning on Macs
Transformers	Python library	Research, custom pipelines, fine-tuning
vLLM · SGLang	Production serving	High-throughput, multi-user GPU serving

Takeaway — apps sit on top of runtimes: LM Studio and Ollama both build on llama.cpp-style engines.

Local AI architecture



Optional add-ons

Local embeddings · local vector database · document ingestion · local RAG · local tools · app integrations

Fully offline

Inference · embeddings · vector search and RAG over local files

May go online

Model downloads and updates · telemetry · web tools · remote API fallback

Takeaway — “local” is a privacy boundary. Check each app’s telemetry and fallback settings before trusting it with sensitive data.

How much memory a local model needs

Memory ≈ parameters × bytes per weight, plus room for context (KV cache).

Model size	16-bit	8-bit	4-bit (GGUF)	Typically fits on
3–4B	~8 GB	~4 GB	~2–3 GB	Most laptops, CPU-only machines, some phones
7–8B	~16 GB	~8 GB	~5 GB	16 GB laptops, entry-level GPUs
13–14B	~28 GB	~14 GB	~8–9 GB	16 GB+ Macs, 12–16 GB GPUs
30–32B	~64 GB	~32 GB	~18–20 GB	24 GB GPUs, 32 GB+ Macs
70B	~140 GB	~70 GB	~40 GB	48 GB+ GPUs, 64 GB+ Macs, workstations

Takeaway — 4-bit quantization is the usual sweet spot. Weights-only figures: leave headroom for long context.

THE COMPLETE AI ECOSYSTEM

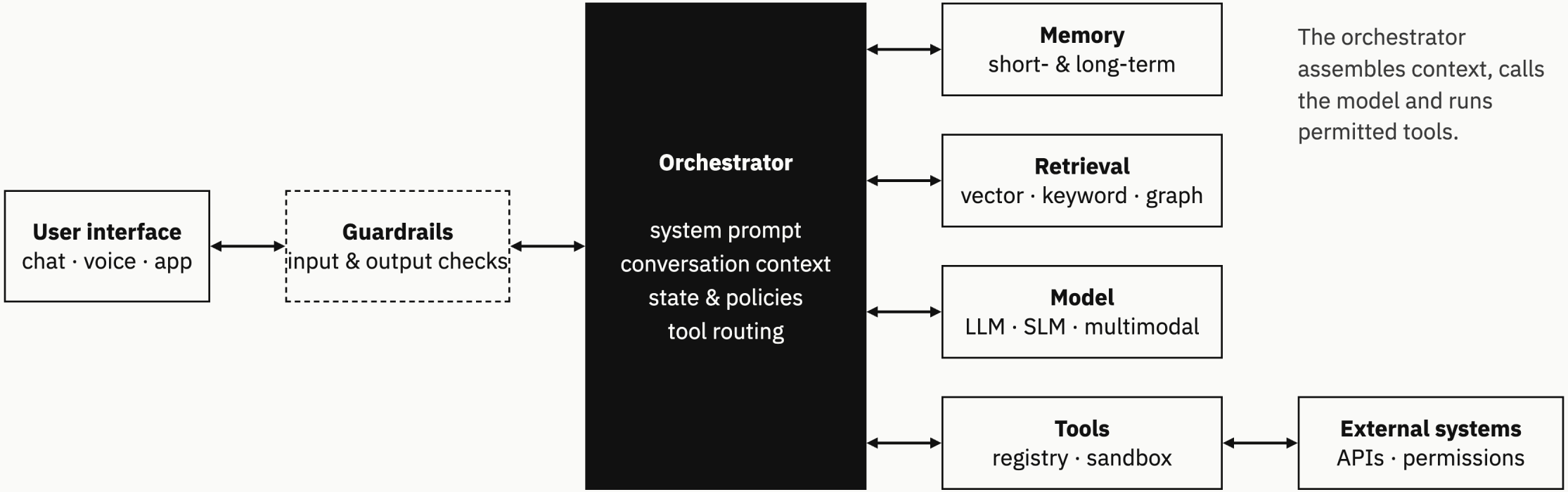
SECTION 05 · LAYERS 07, 13, 14

Applications, RAG & Agents

How models become products that people and systems actually use.

App architecture · App types · RAG · Agents · Connected systems · People

Inside an AI application



The orchestrator assembles context, calls the model and runs permitted tools.

Takeaway — the model is one box of many. Quality and safety come from how the orchestrator, retrieval and guardrails are built.

Not every chatbot is an agent; not every app is RAG

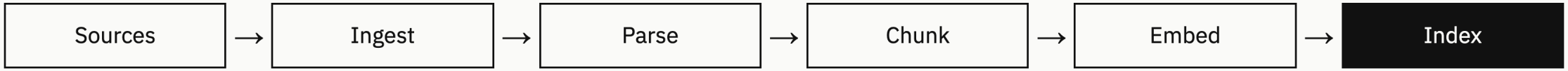
Each step adds one capability, and with it more power and more risk.

Type	Adds	Example	Takes actions?
Chatbot	Model, system prompt, conversation	FAQ assistant	No
RAG app	Retrieval and citations	Policy Q&A over company docs	No
Copilot	In-product context and suggestions	Code completion in an IDE	No — the person acts
Agent	Tools, planning, an action loop	Resolves a support ticket end to end	Yes, within permissions
Multi-agent	Several agents and a coordinator	Research, draft, review pipeline	Yes, coordinated

Takeaway — choose the simplest type that solves the problem. Autonomy should be earned through evaluation, not assumed.

The Retrieval-Augmented Generation pipeline

INDEXING · AHEAD OF TIME



QUERY TIME · EVERY REQUEST



Takeaway — most RAG failures are retrieval failures. If the right passage never reaches the model, no model can answer well.

Techniques that make retrieval work

Dense retrieval
Embeddings match meaning, not words

Sparse retrieval
Keyword scoring for exact terms

Hybrid search
Dense and sparse results combined

Metadata filters
Limit by date, source or permission

Query rewriting
Clarify the question; run variants

Reranking
A second model reorders top hits

Graph retrieval
Follows entity links across hops

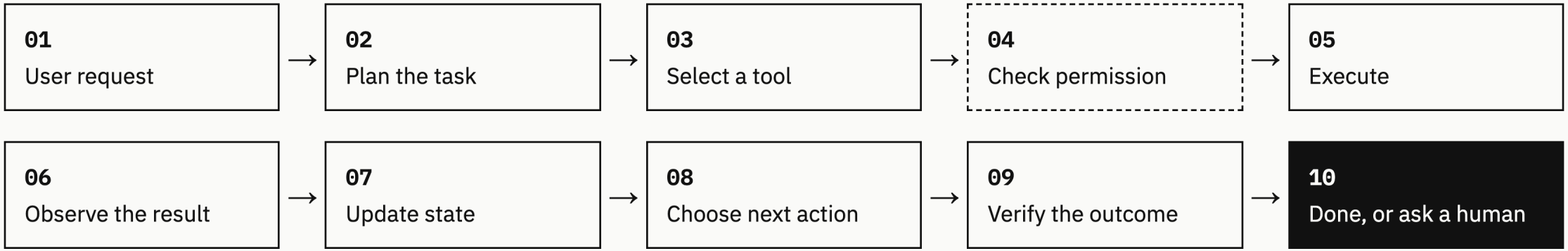
Compression
Trims context to what matters

Citation checks
Each claim backed by a source

Retrieval evals
Recall, precision, faithfulness

Takeaway — measure retrieval separately from generation. A labeled set of questions and correct sources is the most valuable RAG asset.

The agent loop



↺ Steps 02–08 repeat until the goal is verified or a budget, time or iteration limit is reached.

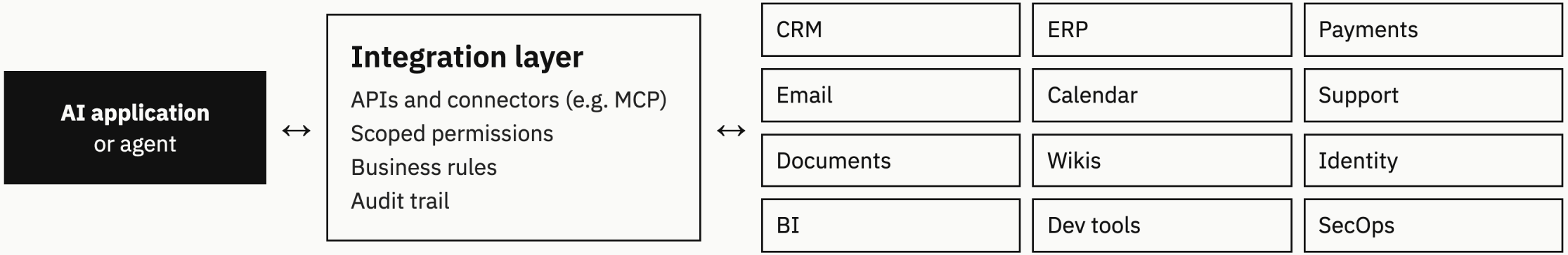
Takeaway — the permission check (04) and verification (09) are what separate a useful agent from a risky one.

What a production agent needs

Component	Why it matters
Function calling, tool registry	The model chooses only from declared, typed tools
Scoped permissions	Least privilege per tool and per user
Sandboxed execution	Generated code runs isolated from real systems
Human approval	Required for irreversible or high-impact actions
Memory and state	Resume work and carry context across steps
Workflow graphs	Fixed steps where reliability matters more than freedom
Retries, timeouts, budgets	Caps on iterations, spend and time
Audit logging	Every plan, tool call and result recorded
Evaluation and red teaming	Test task success and misuse before release

Takeaway — give agents the narrowest permissions that still get the job done, and log everything they do.

Connected systems and enterprise applications



Across industries — software engineering · education · healthcare · finance · manufacturing · retail & e-commerce · government & public services

Takeaway — the AI should act with the user's permissions, never more, and every write to a business system should leave an audit trail.

§5 APPLICATIONS · LAYER 14

People, users and industries

Builders
Software engineers · AI/ML engineers · data scientists · data engineers · researchers

Business
Business analysts · product managers · owners and executives

Domain teams
Healthcare · finance · manufacturing · security · customer support · public sector

Everyone
Students and educators · everyday users

HOW PEOPLE SHAPE AI

Requirements
Define what a good result is

Feedback
Ratings and corrections feed Layer 09

Expertise
Labels, evals and edge cases

Oversight
Approvals and accountability

SECTION 06 · LAYERS 08–11

Production & Infrastructure

Serving models reliably, watching them, and the compute underneath.

Deployment · Serving · MLOps · Observability · Compute · Compute stack · Developer tools

Deployment and inference: where models run

Exposed through REST or GraphQL APIs, streaming or batch, behind authentication, rate limits and quotas.

Option	Runs in	Choose when
Cloud inference API	Provider data centers	Fastest start; frontier models
Self-hosted in cloud	Your cloud GPUs	Open models, control, steady high volume
On-prem, private cloud	Your data center	Data residency or strict regulation
Local PC, workstation	The user’s machine	Privacy, offline use, prototyping
Edge and mobile	Phones, cameras, vehicles	Low latency or no connectivity

Takeaway — many teams mix options: a cloud API for hard tasks, a small self-hosted or local model for cheap, private or offline ones.

Efficient serving, and why training is different

Batching
Share GPU passes

Caching
Reuse prompts, results

Quantization
Fewer bits per weight

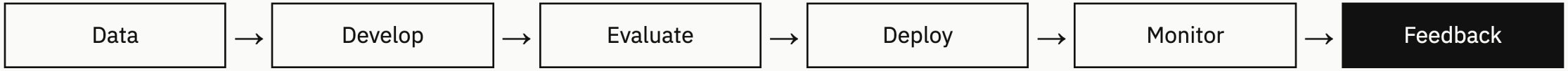
Routing
Small model first

Autoscaling
Load-balanced replicas

Fallbacks
Backup model on failure

	Training infrastructure	Inference infrastructure
Goal	Throughput over long jobs	Low latency per request
Hardware	Multi-node GPU clusters, fast interconnect	Right-sized GPUs, CPUs or edge chips
Load	Scheduled runs lasting days or weeks	Always on, follows user traffic
Bottleneck	Compute and interconnect	Memory and the KV cache
Cost lever	Utilization, checkpointing	Batching, caching, quantization

MLOps and LLMOps: the continuous improvement loop



↺ Feedback returns to Data — the loop never ends while the system is live.

Build

CI/CD · infrastructure as code · versioned models, datasets, prompts, configs

Evaluate

Offline and online evals · regression tests · factuality checks

Release

Canary deployments · A/B tests · fast rollbacks

Respond

Incident response · model retirement and replacement

Takeaway — version prompts like code. In LLM apps a one-line prompt change can shift quality as much as a new model.

What to observe in production

Signal	What to track	Why
Latency	Time to first token, total response time	User experience
Throughput	Requests and tokens per second	Capacity planning
Cost	Tokens and GPU-hours per task	Unit economics
GPU use	Memory and compute utilization	Right-sizing
Quality	Eval scores, hallucination checks	Catch regressions
Retrieval	Hit rate, source relevance	RAG health
Drift	Shifts in inputs and outputs	Retraining triggers
Feedback	Ratings, corrections, escalations	Ground truth
Traces	Prompts, tool calls, responses	Debugging, audits

Takeaway — track cost per successful task, not just cost per token; a cheap model that fails often is expensive.

Compute, cloud, hardware and infrastructure

Cloud providers

AWS · Microsoft Azure · Google Cloud · Oracle Cloud · GPU clouds such as CoreWeave

Compute

CPUs · GPUs · TPUs · NPUs · other accelerators · multi-GPU, multi-node clusters

Vendors

NVIDIA · AMD · Intel · Google TPU · Apple Silicon · AWS Trainium · Cerebras

Platform

VMs · Docker containers · Kubernetes · serverless · storage · networking · load balancers

Security

IAM · secrets management · backup and disaster recovery

Operations

Autoscaling · monitoring · cost management and FinOps

Takeaway — GPU capacity is often the scarcest resource. Plan availability, region and cost before choosing a model size.

The AI compute stack, from silicon to application

Applications	APIs · apps · agents
Model serving	vLLM · SGLang · NVIDIA Triton · Ollama
Inference runtimes	TensorRT · ONNX Runtime · OpenVINO · MLX · llama.cpp
ML frameworks	PyTorch · JAX · TensorFlow
Compute platforms	CUDA (NVIDIA) · ROCm (AMD) · Metal (Apple) · oneAPI (Intel)
Drivers and firmware	Vendor GPU and NPU drivers
Hardware	CPU · GPU · TPU · NPU

Takeaway — each layer depends on the one below. A runtime that lacks support for your hardware rules out that hardware.

AI tools and the developer ecosystem

Category	Examples
Environments	Python · Jupyter · VS Code · Git
Hugging Face	Hub · Transformers · Datasets · PEFT · Accelerate
Frameworks	PyTorch · TensorFlow · JAX
Experiments	MLflow · Weights & Biases
Orchestration	Airflow · Kubeflow · Prefect · Dagster
App frameworks	LangChain · LangGraph · LlamaIndex · Haystack
Evaluation	lm-evaluation-harness · Ragas · DeepEval

Category	Examples
Prompts	MLflow · Langfuse prompt registries
Vector search	Qdrant · Milvus · Weaviate · pgvector · FAISS
Annotation	Label Studio · Argilla
Synthetic data	Distilabel
Serving	vLLM · SGLang · Triton · Ray Serve · BentoML
Observability	Arize Phoenix · Langfuse · OpenTelemetry
Security testing	garak · PyRIT · promptfoo

Takeaway — tools change fast; the categories don’t. Choose by category need first, then compare current tools.

SECTION 07 · LAYER 15

Trust, Security & Economics

The controls that span every layer, and what the whole system costs.

Governance · Security and red teaming · Economics and sustainability

Governance is a control layer, not a final step

Every stage of the lifecycle carries its own controls.

Stage	Controls
Data · 01–04	Privacy and consent, PII redaction, licensing, provenance, retention
Models · 05, 06, 12	Fairness and bias tests, model documentation, license checks, model risk review
Applications · 07, 13	Prompt-injection defenses, content safeguards, least privilege, human oversight
Infrastructure · 08, 10	Identity and access, encryption, secrets management, vendor risk, energy use
Operations · 09	Audit trails, AI incident response, monitoring, regulatory compliance
People · 14	Transparency, explainability, accessibility, training for users

Takeaway — retrofitting governance costs far more than designing it in. Assign an owner for each control at each stage.

AI security threats and defenses

Threat	What happens	Defense
Prompt injection	Input overrides instructions	Separate instructions from data; least privilege
Indirect injection	Hidden orders in retrieved content	Treat retrieved text as data; tool allow-lists
Jailbreaks	Users bypass safety policies	Safety training, classifiers, red teaming
Data leakage	Secrets or PII in outputs	Redaction, access-aware retrieval, scanning
Poisoning	Tampered training or retrieval data	Provenance, validation, trusted sources
Model extraction	Queries used to copy a model	Rate limits, abuse monitoring
Agent misuse	Harmful tool calls, untrusted code	Permissions, approval, sandboxes
Supply chain	Unsafe model files or packages	Trusted registries, safetensors, scanning
Denial of service	Costly prompts exhaust capacity	Quotas, timeouts, input limits

AI economics: cloud versus self-hosted cost

Drivers: training · inference · tokens · GPU use · storage · network · energy · licenses

	Cloud API	Self-hosted or local
Upfront	None	Hardware or reserved GPUs
Per use	Price per token	Electricity and operations time
Scaling	Instant and elastic	Limited by capacity you own
Utilization	Pay only for what you use	Idle hardware still costs money
Licensing	Included in the price	Model license terms apply
Best when	Spiky volume, top quality	Steady high volume, privacy, offline

Takeaway — compare cost per successful task. A larger model that succeeds first time can beat a cheaper one that needs retries.

SECTION 08

Frontier & Reference

What is emerging, two end-to-end architectures, and a glossary.

Emerging AI · Enterprise reference architecture · Local reference architecture · Glossary

Emerging and specialized AI, by maturity

IN PRODUCTION

Established

- On-device AI
- Time-series intelligence
- Graph learning
- Geospatial AI
- AI-assisted coding
- Federated learning

EARLY DEPLOYMENTS

Emerging

- Robotics and embodied AI
- Scientific AI
- AI for drug discovery
- Privacy-preserving ML
- Simulation for training
- Autonomy in set domains

RESEARCH

Speculative

- General world models
- Open-ended autonomous agents
- General-purpose robots

Takeaway — judge new categories by evidence of deployment, not demos. Maturity labels shift every year.

§8 REFERENCE ARCHITECTURE A

An enterprise knowledge assistant, layer by layer

Layer	Choice in this design
01 Sources	Wikis, PDFs, support tickets, CRM records
02–04 Prepare	Nightly sync plus CDC, parsing, chunking, PII redaction, permission tags
03 Store	Object storage for files; hybrid vector and keyword index
12 Models	Proprietary API for hard questions; small open model for routing
07 Application	RAG with citations; ticket-creation tool behind human approval
08 Serving	API gateway, streaming responses, caching, rate limits
13 Connected	SSO identity, CRM and ticketing via connectors, audit log
09 Operations	Traces, retrieval evals, feedback buttons, cost per answer
15 Governance	Access-aware retrieval, injection defenses, retention, incident runbook

Takeaway — users only ever see answers drawn from documents they are already allowed to open.

§8 REFERENCE ARCHITECTURE B

A private assistant that runs on one laptop

Hardware
32 GB Apple Silicon Mac or 12 GB+ GPU

→

Runtime
Ollama or LM Studio local API

→

Models
7–14B chat model at 4-bit, local embeddings

→

Knowledge
Local vector store over your folders

→

Interface
Desktop chat with document upload

Stays on the device
Prompts, documents, embeddings, answers and chat history

Decide deliberately
Model downloads, app updates, telemetry, web search, cloud fallback

Takeaway — local trades peak quality for privacy and zero per-token cost; it suits drafting, search and summarizing.

§8 REFERENCE

Glossary

Term	Meaning
Token	A word piece; the unit models read and bill by
Embedding	A vector capturing meaning, used for search
Context window	How many tokens one request can hold
KV cache	Memory of processed tokens during generation
Quantization	Storing weights in fewer bits to save memory
GGUF	File format for quantized local models

Term	Meaning
LoRA, QLoRA	Fine-tuning small adapter weights only
RAG	Retrieving documents to ground an answer
Reranker	Model that reorders search results by relevance
Hallucination	Fluent output that is not supported by facts
Agent	A model that plans and acts with tools in a loop
MCP	Model Context Protocol, an open connector standard

Five things to remember

01	Data quality and permissions decide most outcomes.
02	Start light: prompting, then RAG, then fine-tuning.
03	Open-weight is a license; local is a location.
04	Agents need narrow permissions, limits and verification.
05	Governance and feedback run through every layer, all the time.